The Mathematics of tagd

A Formal Description of the Mathematical Structures
in the tagd Semantic-Relational System

Isaac N Colley

April 2026

Abstract

The tagd system was built from engineering instincts about how knowledge should be structured. This paper makes the mathematics of those instincts explicit. Every claim here is grounded in the tagd source code and the TAGL language specification. The core thesis is simple: *information is structural*. A tag's meaning is not intrinsic — it is determined entirely by its position in the tagspace, by its relations to other tags. This is not a metaphor. It is a precise mathematical claim, and the sections below give it precise mathematical form, identifying in the source code the structures of set theory, partial orders, and Alexandrov topology.

Contents

1 Primitives **2**
1.1 Tags . . . 2
1.2 The Root and the Axiom . . . 2

2 The Subordinate Relation and the Rooted Tree **2**
2.1 Subordinate Relations . . . 2
2.2 The Tagspace as a Rooted Tree . . . 3

3 Rank: Encoding the Tree in Strings **3**
3.1 The Rank Function . . . 3
3.2 The Prefix Partial Order . . . 4
3.3 Verifying the Partial Order Axioms . . . 4
3.4 Lexicographic Total Order . . . 5

4 The Tagspace Topology **5**
4.1 From Partial Order to Topology . . . 5
4.2 Verifying the Topology Axioms . . . 5
4.3 Specialization Order . . . 5
4.4 Continuity as Meaning-Preservation . . . 6

5 The Hard Tag Base: A Universal Subspace **6**
5.1 Hard Tags . . . 6
5.2 The Hard Tag Subspace . . . 6
5.3 Interoperability via the Common Base . . . 6

6 Predicates: Fibered Structure **7**
6.1 Predicates as Horizontal Relations . . . 7
6.2 The Two-Layer Structure . . . 7
6.3 Relators Are Tags . . . 7

7 Structural Identity **7**
7.1 Identity Is Position 7
7.2 The Canonical Form 8

8 Summary **8**

9 Conclusion **8**

1 Primitives

1.1 Tags

A **tag** is the atomic unit of the tagd system. All tokens in TAGL are UTF-8 labels [6]. A tag carries two distinct identities:

- **Nominal identity**: a globally unique UTF-8 label.
- **Structural identity**: its position in the tagspace hierarchy, defined by its subordinate relation and predicate list [5].

Definition 1.1 (Tag Set)**.** *Let T be the set of all tags in a tagspace. Each tag $t \in T$ is identified by a string* $\mathrm{id}(t) \in \Sigma^*$*, where Σ^* denotes the set of all finite UTF-8 strings.*

In TAGL, defining a tag looks like:

```
>> dog _is_a mammal;
```

Listing 1: Defining a tag via a subordinate relation

Here `dog` is a tag. Its nominal identity is the string `"dog"`. Its structural identity is defined by its subordinate relation to `mammal`.

1.2 The Root and the Axiom

Every tagspace has a distinguished root tag: `_entity`. The README states:

> `_entity` is axiomatic and self-referencing (i.e. `_entity _sub _entity`). It is the only tag having this relation. It is the root of a tagspace.

Definition 1.2 (Axiomatic Root)**.** *The tag `_entity` is the **axiomatic root** of every tagspace. It is the unique fixed point of the subordinate relation: the one tag whose super-object is itself.*

In TAGL notation:

```
_entity _sub _entity
```

Listing 2: The axiomatic root

In set-theoretic terms, `_entity` is the **least element** of the tagspace under the subordinate order — the tag that is an ancestor of everything, including itself.

2 The Subordinate Relation and the Rooted Tree

2.1 Subordinate Relations

A tag's structural identity is defined by exactly one subordinate (sub) relation. The TAGL specification states [5]:

A subject MUST have exactly one subordinate relation to exist in the tagspace.

The sub relators are defined in `hard-tags.h` [1]:

```
#define HARD_TAG_SUB     "_sub"     // most abstract sub relator
#define HARD_TAG_IS_A    "_is_a"    // instance subordination
#define HARD_TAG_TYPE_OF "_type_of" // type subordination
```

Listing 3: Sub relator definitions in hard-tags.h

A typical TAGL knowledge hierarchy:

```
>> physical_object -^ _entity;
>> living_thing _type_of physical_object;
>> animal _type_of living_thing;
>> mammal _type_of animal;
>> dog _is_a mammal;
```

Listing 4: Building a subordinate hierarchy in TAGL

2.2 The Tagspace as a Rooted Tree

The subordinate relation organizes all tags into a **rooted tree**.

Definition 2.1 (Tagspace Tree)**.** *A tagspace is a pair* (T, p) *where:*

- T *is a set of tags containing the distinguished root* `_entity`*,*
- $p : T \setminus \{\texttt{_entity}\} \to T$ *is the* parent function*, assigning each non-root tag its super-object.*

There are no cycles (except the axiomatic self-reference of `_entity`*).*

The tree structure of the example above:

```
_entity
  physical_object
    living_thing
      animal
        mammal
          dog
```

This is not merely a data structure choice. The tree encodes *taxonomic knowledge*: to be a `dog` is to be a `mammal`, which is to be an `animal`, and so on. Structural position *is* meaning.

3 Rank: Encoding the Tree in Strings

3.1 The Rank Function

Each tag is assigned a **rank** — a UTF-8 byte string encoding its position in the tagspace tree. The `rank.h` source states:

> A rank string is the ordinal value of each node in the tagspace tree. A code point at a particular level is known as the rank value.

Definition 3.1 (Rank Function)**.** *The rank function* $r : T \to \Sigma^*$ *assigns to each tag a UTF-8 string such that the string's structure reflects the tag's position in the tree.*

Concrete rank assignments from the bootstrap dump [7]:

Tag	Rank (dotted)	Position
_entity	(empty)	level 0, root
_sub	1	level 1
_is_a	1.1	level 2
_type_of	1.2	level 2
_rel	2	level 1
_has	2.1	level 2
_number	3	level 1
_integer	3.1	level 2
_float	3.2	level 2
_interrogator	4	level 1

3.2 The Prefix Partial Order

The key property of ranks is stated explicitly in `rank.h`:

> All children of a particular node in the rank hierarchy will be prefixed by the parent node — that is, rank substrings define subtrees in the hierarchy.

Definition 3.2 (Containership Relation)**.** *Define the* ***containership relation*** $\sqsubseteq$ *on* T *by:*

$$a \sqsubseteq b \iff r(b) \text{ is a prefix of } r(a).$$

That is, $a \sqsubseteq b$ *means* b *is an ancestor of* a *(or* $b = a$*). Tag* b contains *tag* a *in the hierarchy.*

This relation is implemented directly in `rank.h`:

```
// this rank contains the rank being compared,
// in other words, this rank is a prefix to the other
bool contains(const rank&) const;
```

Listing 5: Containership via rank prefix in rank.h

And used in `tagd.cc` for structural operations on tag sets:

```
if (b->rank().contains(a->rank())) { // rank can contain itself (b == a)
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 6: Containership used in merge_containing_tags (tagd.cc)

3.3 Verifying the Partial Order Axioms

Theorem 3.1. *The containership relation* $\sqsubseteq$ *is a partial order on* T*.*

Proof. We verify the three required properties.

Reflexivity. `rank.h` notes that `contains()` holds when $b = a$, since every string is a prefix of itself. Hence $a \sqsubseteq a$ for all $a \in T$.

Antisymmetry. Suppose $a \sqsubseteq b$ and $b \sqsubseteq a$. Then $r(b)$ is a prefix of $r(a)$ and $r(a)$ is a prefix of $r(b)$. A string u is a prefix of v and v is a prefix of u if and only if $u = v$. Hence $r(a) = r(b)$, which implies $a = b$ since ranks uniquely identify tags.

Transitivity. Suppose $a \sqsubseteq b$ and $b \sqsubseteq c$. Then $r(b)$ is a prefix of $r(a)$ and $r(c)$ is a prefix of $r(b)$. Since the prefix relation on strings is transitive, $r(c)$ is a prefix of $r(a)$, so $a \sqsubseteq c$. □

Therefore $(T, \sqsubseteq)$ is a **poset** (partially ordered set).

3.4 Lexicographic Total Order

Ranks also carry a total **lexicographic order**, implemented in `rank.h`:

```
bool operator<(const rank& rhs) const { return _data < rhs._data; }
```

Listing 7: Lexicographic ordering of ranks

This order — a deliberate UTF-8 design choice noted in `rank.h` ("it collates lexicographically") — enables efficient storage and retrieval of the entire tagspace as a sorted structure. Tags within the same subtree sort together. The tree *is* the index.

4 The Tagspace Topology

4.1 From Partial Order to Topology

Every partial order canonically generates a topology. Given the poset $(T, \sqsubseteq)$, we define:

Definition 4.1 (Tagspace Open Sets)**.** *A subset $U \subseteq T$ is **open** if it is upward-closed under $\sqsubseteq$:*

$$U \in \mathcal{T} \iff \forall\, a \in U,\ \forall\, b \in T:\ a \sqsubseteq b \Rightarrow b \in U.$$

In the tagspace tree, "upward-closed" means: if a tag is in the set, all its ancestors are too. Each basic open set is the **subtree rooted at a tag** together with all tags above it.

This construction is known as the **Alexandrov topology** — the topology generated by a partial order in which arbitrary intersections (not just finite ones) of open sets remain open.

4.2 Verifying the Topology Axioms

Theorem 4.1. *Let M be the set of all tags in a tagspace and let $\mathcal{T}$ be the collection of upward-closed subsets under $\sqsubseteq$. Then $(M, \mathcal{T})$ is a topological space.*

Proof. We verify the three axioms.

(i) $\emptyset \in \mathcal{T}$ and $M \in \mathcal{T}$. The empty set is vacuously upward-closed. The full set M is upward-closed because there is no $b \in T$ outside M. Both belong to $\mathcal{T}$.

(ii) Finite intersections. Let $U, V \in \mathcal{T}$. Suppose $a \in U \cap V$ and $a \sqsubseteq b$. Then $b \in U$ (since U is upward-closed) and $b \in V$ (since V is upward-closed), so $b \in U \cap V$. Hence $U \cap V \in \mathcal{T}$.

(iii) Arbitrary unions. Let $\{U_\alpha\}$ be any collection of sets in $\mathcal{T}$. Suppose $a \in \bigcup_\alpha U_\alpha$ and $a \sqsubseteq b$. Then $a \in U_\alpha$ for some α. Since U_α is upward-closed, $b \in U_\alpha \subseteq \bigcup_\alpha U_\alpha$. Hence $\bigcup_\alpha U_\alpha \in \mathcal{T}$. □

Therefore $(M, \mathcal{T})$ is a **topological space**. The `merge_containing_tags` function in `tagd.cc` computes exactly the intersection of two subtrees, exhibiting axiom (ii) in code:

```
if (b->rank().contains(a->rank())) {
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 8: Intersection of subtrees in tagd.cc

4.3 Specialization Order

In the Alexandrov topology, every topological space carries a natural **specialization preorder**: a specializes b (written $a \rightsquigarrow b$) when a is in the topological closure of $\{b\}$.

Proposition 4.1. *In the tagspace topology, the specialization order coincides with the subordinate order: $a \rightsquigarrow b$ if and only if a is a descendant of b in the tagspace tree.*

In plain language: `dog` specializes `mammal`, which specializes `animal`. More specific tags (deeper in the tree) specialize more general ones. Taxonomic inheritance is not a rule to be enforced — it is a geometric consequence of the shape of the space.

4.4 Continuity as Meaning-Preservation

Definition 4.2 (Tagspace Map)**.** *A map $f : T_1 \to T_2$ between two tagspaces is* ***topologically continuous*** *if it preserves open sets: $f^{-1}(U) \in \mathcal{T}_1$ for every $U \in \mathcal{T}_2$.*

Proposition 4.2. *A map $f : T_1 \to T_2$ is continuous (with respect to the Alexandrov topologies) if and only if it is order-preserving: whenever $a \sqsubseteq b$ in T_1, we have $f(a) \sqsubseteq f(b)$ in T_2.*

This gives a precise mathematical definition of a **meaning-preserving transformation** between tagspaces. If `dog _is_a mammal` in T_1, a continuous map must send `dog` somewhere below the image of `mammal` in T_2.

5 The Hard Tag Base: A Universal Subspace

5.1 Hard Tags

Every tagspace — no matter how different its user-defined tags — contains exactly the same set of hard tags, defined in `hard-tags.h` [1]:

```
#define HARD_TAG_ENTITY       "_entity"       // root
#define HARD_TAG_SUB          "_sub"          // subordination
#define HARD_TAG_IS_A         "_is_a"         // instance relation
#define HARD_TAG_TYPE_OF      "_type_of"      // type relation
#define HARD_TAG_RELATOR      "_rel"          // predicate root
#define HARD_TAG_HAS          "_has"          // possession
#define HARD_TAG_CAN          "_can"          // capability
#define HARD_TAG_NUMBER       "_number"       // numeric type root
#define HARD_TAG_INTEGER      "_integer"      // integer type
#define HARD_TAG_FLOAT        "_float"        // float type
#define HARD_TAG_INTERROGATOR "_interrogator" // query root
```

Listing 9: Core hard tag definitions in hard-tags.h

5.2 The Hard Tag Subspace

Definition 5.1 (Hard Tag Subspace)**.** *Let $H \subset T$ denote the set of hard tags in any tagspace T. The induced subspace topology on H is identical in every tagspace. H is therefore the* ***universal invariant subspace*** *— a fixed topological base that every tagspace shares by construction.*

Remark 5.1. *In categorical terms, the hard tag subspace is the* ***initial object*** *in the category of tagspaces: every tagspace is an extension of H, and every valid map between tagspaces must fix H.*

5.3 Interoperability via the Common Base

Because every tagspace contains H as a common topological foundation, any two tagspaces can communicate. The hard tags are the shared language — the invariant ground that makes the `httagd` web protocol possible. Two tagspaces may diverge arbitrarily in their user-defined tags, but they always agree on the structure of H.

```
-- invariant in every tagspace:
_entity _sub _entity

-- user-defined, varies by tagspace:
>> dog _is_a mammal;
```

Listing 10: Hard tags are invariant; user tags vary

6 Predicates: Fibered Structure

6.1 Predicates as Horizontal Relations

Predicates are relations attached to a tag that do not alter its position in the tree. From `TAGL-spec.md` [5]:

> A predicate consists of a relator followed by an object_list. A relator MUST be a tag subordinate to `_rel`.

```
>> dog _is_a mammal     -- vertical: places dog in the tree
    _has legs = 4, tail
    _can bark;          -- horizontal: predicate data at dog
```

Listing 11: Vertical identity and horizontal predicates

6.2 The Two-Layer Structure

The tagspace carries two distinct mathematical layers:

Definition 6.1 (Two-Layer Tagspace Structure). *1. **Base space**: the topological space $(M, \mathcal{T})$ defined by subordinate relations and ranks — the vertical dimension.*

*2. **Fiber**: at each tag $t \in M$, a set of predicates $P(t) \subseteq \mathrm{Rel} \times T \times \mathrm{Mod}$ — the horizontal dimension, where* Rel *is the set of relator tags and* Mod *is the set of modifier values.*

This two-layer structure — a base space with additional data attached at each point — is analogous to a **fiber bundle** in differential geometry. The topology gives the skeleton; the predicates give the flesh.

The `abstract_tag` class in `tagd.h` encodes exactly this decomposition:

```
class abstract_tag {
    tagd::rank    _rank;        // position in the base space
    predicate_set relations;    // fiber: additional data at this point
};
```

Listing 12: Two-layer structure in abstract_tag (tagd.h)

6.3 Relators Are Tags

Crucially, relators are themselves tags — subordinate to `_rel` in the tagspace tree. This means predicates are not outside the system; they are *inside* it. The tagspace is self-describing.

From the bootstrap dump [7]:

```
_rel     rank: 2        (root of all relators)
_has     rank: 2.1
_can     rank: 2.2
_refers  rank: 2.3
```

The language used to attach meaning to tags is itself expressed in the same structural language.

7 Structural Identity

7.1 Identity Is Position

The deepest mathematical claim of the tagd system is that a tag's identity is entirely structural. The TAGL specification defines:

> **Structural identity**: the identity of a tag defined by its subordinate relation (rank) and predicate_list.

A tag `dog` is not `dog` because of the string `"dog"`. It is `dog` because of where it sits — under `mammal`, under `animal`, under `living_thing`, under `_entity` — and because of what predicates are attached to it there. Change the position (the subordinate relation), and you have a different tag, regardless of the label.

This is **mathematical structuralism** applied to knowledge representation: identity is relational, not intrinsic.

7.2 The Canonical Form

A tagspace can be fully serialized as TAGL text via `dump()`. From the TAGL Thesis document:

> The canonical form of a tagspace is its dump — the UTF-8 TAGL code that represents the tagspace (and can be loaded).

The dump is not a record of a structure. It *is* the structure, expressed in TAGL. The tagspace is its own specification.

8 Summary

Table 1 collects the mathematical structures identified in the tagd source and maps each to its realization in code and TAGL.

Mathematical Structure	**tagd Realization**	**Source**
Set	All tags T in a tagspace	`tagd.h: tag_set`
Rooted tree	Subordinate relations from `_entity`	`tagd-README.md`
Partial order (poset)	Rank prefix containership $\sqsubseteq$	`rank.h: contains()`
Lexicographic total order	Rank byte string comparison	`rank.h: operator<`
Alexandrov topology	Upward-closed subtrees as open sets	`rank.h`, `tagd.cc`
Universal base subspace	Hard tag set H	`hard-tags.h`
Fiber / attached structure	Predicate sets $P(t)$ at each tag	`tagd.h: predicate_set`
Continuous map	Order-preserving tagspace map	`tagd.cc: merge_containing_tags`
Canonical serialization	`dump()` in TAGL canonical form	`TAGL-spec.md`

Table 1: Mathematical structures and their tagd source realizations.

9 Conclusion

The tagd system is not merely a database or a knowledge graph. It is a **topological space of structured meaning**. Tags are points. Subordinate relations give the space its shape. Ranks encode that shape as a partial order. The Alexandrov topology makes the shape mathematically precise. Predicates attach additional structure at each point as a fiber over the base topology. And the hard tag base ensures that every tagspace shares the same foundational geometry.

TAGL is the language for acting on this space: defining points, querying regions, transforming structure. Every TAGL statement is a topological operation.

The vision that *information is structural* — that meaning comes from shape, not from labels — is not a design philosophy. It is a theorem about how the system works.

References

[1] tagd source code, `tagd/include/tagd/hard-tags.h`. Hard tag definitions and axioms.

[2] tagd source code, `tagd/include/tagd/rank.h`. Rank structure, prefix order, and `contains()` implementation.

[3] tagd source code, `tagd/include/tagd.h`. `abstract_tag`, `predicate_set`, and tag set operations.

[4] tagd source code, `tagd/src/tagd.cc`. `merge_containing_tags` and merge operations.

[5] *TAGL Specification*, `TAGL-spec.md`. Language specification and semantic model.

[6] *tagd README*, `tagd-README.md`. Working examples and TAGL tutorial.

[7] *Bootstrap Dump Grid*, `bootstrap-dump-grid.txt`. Concrete tagspace rank assignments.